

CHAPTER 1

INTRODUCTION

The automation has a dominant role in today's world, it is an ever- evolving field with wider range of application. These days most of industrial and daily routine works are being implemented through automation via robots. Robots are great tool to mankind. They are used to do repetitive, time – consuming, delicate and dangerous jobs. It increases speed and quality of work and completes the tasks even more precisely and efficiently than humans. The project proposes robotic vehicle which perform tasks such as object detection, automatic pick and place operations, obstacles detection and avoidance operations[1]. This eliminates the human error and human intervention to get more precise work. Once installed they replaces the workforce in industrial workplaces which reduces the cost of human labor, they make sure to keep installed area clean with zero litter around the area. Good quality serving system also attracting people towards the installed restaurants. Typically a robot mounted on a stand is used to perform pick and place operation. Here it has been replaced to mobile robot which is capable of moving around in surrounding and not fixed in one exact location. That too without any collision, to the human defined locations. In future with advancement of technologies the robot will become even more smarter, more flexible and too efficient[2].

Industrial robots are found in a variety of locations including the automobile and manufacturing industries. Robots cut and shape fabricated parts, assemble machinery and inspect manufactured parts. Some types of jobs robots do: load bricks, die cast, drill, fasten, forge, make glass, grind, heat treat, load/unload machines, machine parts, handle parts, measure, monitor radiation, run nuts, sort parts, clean parts, profile objects, perform quality control, rivet, sand blast, change tools and weld[3].

Outside the manufacturing world robots perform other important jobs. They can be found in hazardous duty service, CAD/CAM design and prototyping, maintenance jobs, fighting fires, medical applications, military warfare and on the farm[4].

1.1 Scope of work

The project on the development of a robotic arm for object sorting with Bluetooth-controlled operation encompasses a range of tasks, from design and development to testing and implementation. The scope of work includes:

1. System Design and Planning:

Develop system architecture, including hardware components, software interfaces, and communication protocols

2. Hardware Development:

Mechanical Design:

Design and fabricate the robotic arm with components such as actuators, servos, linkages and grippers.

Ensure stability, range of motion, and payload capacity suitable for sorting tasks.

3. Software Development:

Programming:

Develop firmware for microcontroller-based control of the robotic arm.

Implement algorithms for object sorting, path planning, and motion control.

1.2 Objectives

The primary objective of this project is to design and implement an automated system for sorting moving objects based on their color and shape using a robotic arm. The system aims to enhance sorting efficiency, accuracy, and speed while minimizing human intervention. The specific objectives include:

1. Object Detection:-Develop a vision-based system capable of identifying and classifying objects by their color and shape in real-time using advanced image processing techniques.

2. Dynamic Tracking:-Implement a mechanism to track objects as they move on a conveyor belt, ensuring accurate positioning for robotic arm operations.

Robotic arm based on object sorting with bluetooth command

3. Robotic Arm Control:-Integrate and program a robotic arm to perform precise pick-and-place tasks based on the classification of objects.
4. Automation and Scalability:-Create a fully automated system that can handle high-speed production lines and adapt to a variety of object types and sizes.
5. Minimizing Errors:-Reduce manual errors by automating the sorting process, ensuring consistent performance and reliability.
6. Cost-Effective Solution:-Design a system that is both efficient and cost-effective, making it suitable for small- and medium-scale industries.
7. Adaptability:-Enable the system to learn and improve performance over time, incorporating machine learning for better adaptability to changing sorting requirements.

1.3 Problem statement

In modern industries, the sorting of objects based on specific attributes, such as color and shape, is a critical operation. Traditional manual sorting methods are labor-intensive, error-prone, and inefficient, especially when dealing with high-speed or large-scale production lines. This creates bottlenecks in the manufacturing process and increases operational costs.

While existing automated systems can address some of these challenges, many are limited in adaptability and struggle to handle dynamic scenarios, such as sorting objects on a moving conveyor belt. These systems often lack real-time processing capabilities and fail to integrate seamlessly with robotic arms for precise object manipulation.

The challenge lies in designing an efficient, cost-effective, and scalable system capable of accurately detecting, classifying, and sorting moving objects based on their color and shape in real-time. This system must incorporate advanced vision-based detection, robust image processing algorithms, and precise robotic arm control to achieve high accuracy and speed in sorting operations.

Chapter 2

LITERATURE SURVEY

[1] “Color based Product Sorting Machine using Raspberry Pi”

Authors: Dr. P. Perumal, Dr. B. Mathivanan, Dr. K. Deepa

Published: 2024 5th International Conference on Smart Electronics and Communication (ICOSEC) | 979-8-3315-0440-3/24/\$31.00 ©2024 IEEE | DOI: 10.1109/ICOSEC61587.2024.10722515

Bottles can be arranged in different groups based on their qualities, 4 IEEE | DOI: 10.1109/ICOSEC61587.2024.10722515 such as size, color, form, etc. This work primarily focuses on designing and developing a bottle-sorting device powered by Arduino. In this instance, the glass's height and color have been used to identify bottles from different companies. Most of the time, this is done by hand. Color and contact sensors are employed to determine the size and color of the bottles. The conveyer belt-transported bottles are sorted by turning on the appropriate product gateway. A rotary actuator with an arm removes the bottles off the belt conveyor for sorting and processing

[2] Design and Development of a Color Sorting Robotic ARM”

Authors: Dr. Sanjay Kumar¹, Shiv Sahay Yadav², Anirudha Kalaskar Pradyumna³, Awate Maitriy Shripal⁴, Nerlekar Mandar Dattaprasad⁵. Published: International Journal of Enhanced Research in Management & Computer Applications ISSN: 2319-7471, Vol. 13, Issue 4, April-2024, Impact Factor: 8.285

The integration of robotic systems in industrial applications has evolved significantly over the decades, driven by advancements in sensor technology, microcontrollers, and automation techniques. This literature review explores the various aspects of robotic arms, particularly focusing on color sorting applications, and highlights the contributions of various researchers in this field. Robotic manipulators have long been employed in industrial settings for tasks that require precision and repeatability

[3] “Simulation of Pick and Place Robotic Arm using Coppeliasim”

Authors: S. Rooban, R Manimegalai,, Published : 2022 6th International Conference on Computing Methodologies and Communication (ICCMC) | 978-1-6654-1028-1/22/\$31.00

Robotic arm based on object sorting with bluetooth command

©2022 IEEE | DOI: 10.1109/ICCMC53470.2022.9754013 To assemble the elements of the robotic system it's essential to undergo diverse structure and methodologies to achieve perfect momentum of designed robotic structure. The design flow of robotic system includes programming the task and the working environment. Robot design includes mechanical units, sensor units, actuation units and supervision units.

[4] “Color Sorting Pick Up and Drop Robotic Arm”

Authors: Aryan Bhapkar, Manasi Pawaskar, Amey Raut Published : 2024 IEEE 9th International Conference for Convergence in Technology (I2CT) | 979-8-3503-9447-4/24/\$31.00 ©2024 IEEE | DOI: 10.1109/I2CT61223.2024.10544011 In their paper, LimJieet al. develop a six-degree-of freedom control system for a robot arm with an emphasis on pick-and-place and automatic color sorting using Blynk apps. The system allows for wireless control and manual color sorting using servomotors and color sensors, producing precise and there is no vibration. The motion of the robotic arm is controlled by Wifi which is connected to the Blynk app allowing the robot to show movement change. The author Yashas M. Aet all present the automated object sorting robot with a Raspberry Pi and Freeduino microcontroller is presented.

[5] “Development of a 3DOF Color Sorting based Robotic Arm using MATLAB GUI”

Authors: Tonusree Mohanto, Aparajita Talukder, Zinat Tasneem Published: 2020 23rd International Conference on Computer and Information Technology (ICCIT) | 978-1-6654-2244-4/20/\$31.00 ©2020 IEEE | DOI: 10.1109/ICCIT51783.2020.9392750 The field of Robotics and Automation occupies most of the industrial world along with household and other adverse human jobs with the desire to synthesize some aspects of human functions by the use of mechanisms, sensors, actuators and computers over the last few decades. Robotic arm is one of the best uses of Robotics Science as it can be compared to human hands and can widely be used in industrial proposes

[6] “Low Cost Object Sorting Robotic Arm using Raspberry Pi”

Authors: Viren Pereira, Vandyk Amsdem Fernandes and Junieta Sequeira Published: 2014 IEEE Global Humanitarian Technology Conference - South Asia Satellite (GHTC-SAS) | September 26-27, 2014 | Trivandrum In this fast growing industrial age every industrial unit

Robotic arm based on object sorting with bluetooth command

needs speed in manufacturing. Robotics has found a wide application in industries. Automation provides far better service to customers eliminating the monotonous work by human, achieving accuracy and speed in work. They are high in demand and are used to carry out most of the work which saves time and is more efficient. The use of Open-Source environment makes it cost effective, Linux based Operating system used in Raspberry Pi (Raspbian OS), and GNU Octave, Python and Arduino IDE are freely available for user to use and also to develop.

[7] “Color Sorting Robotic Arm”

Authors: S. A. Khan, T. Z. Anika, N. Sultana Published: 2019 International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST) this fastest era, time is very precious. To save this valuable time, robotics plays an important role in human life. Robot is a manmade thing which reduces human effort & time. Especially in industrial sector, machine plays a vital role. Machine can produce various products in a very short time. Robotic arm is one kind of machine. It can be used in industrial sector or house hold automation purpose depending on the features, size and also ability of the robotic arm.

[8] “Development of Colour Sorting Robotic Arm Using TCS3200 Sensor”

Authors: Ahmad Rafiq Mohd Khairudin, Muhammad Hanif Abdul Karim, Asma Abu Samah Published: 2021 IEEE 9th Conference on Systems, Process and Control (ICSPC) | 978-1-6654-3763-9/21/\$31.00 ©2021 IEEE | DOI: 10.1109/ICSPC53359.2021.9689114 Robotic arm is a programmable device that uses digital technologies to perform specific tasks in various applications such as manufacturing, fabrication, postal services, or household automation purposes [1, 2]. This robotic manipulator is used in several applications by changing the program in the microcontroller and replacing the end effectors, such as magnetic gripper, vacuum gripper or gripper, with pressure pads depending on the given tasks. It is an ideal logistic robot for industry owners, the elderly, or even employees assigned to laborious tasks, letting them spend more time on other priorities.

Chapter 3

TECHNOLOGY USED

3.1 Block Diagram:-

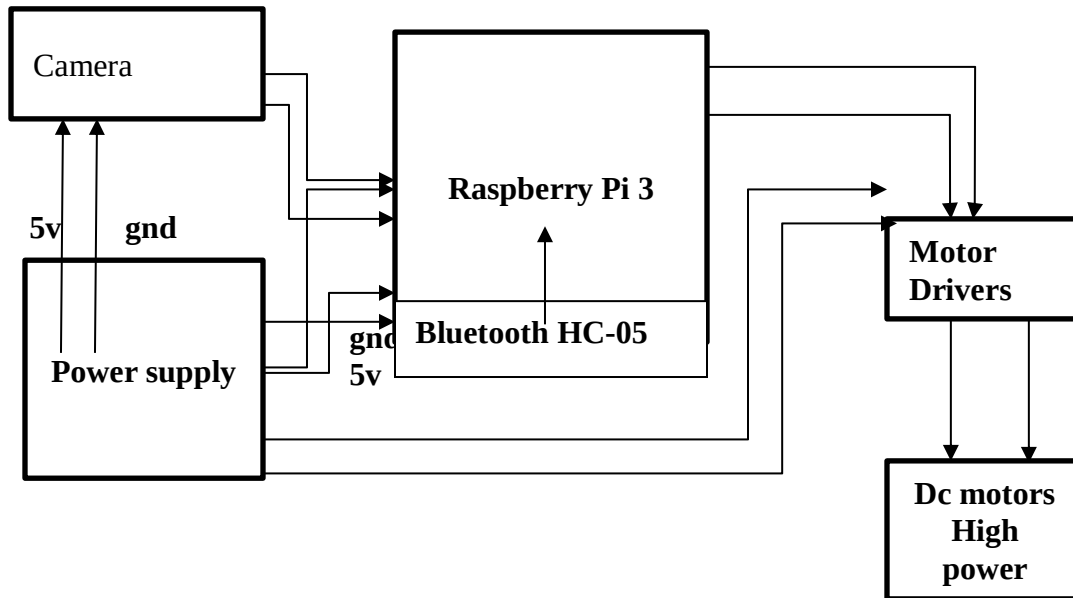


Fig 3.1 Block Diagram

As we follow along with the block diagram. This system is a Raspberry Pi-based robotic control circuit that integrates a camera, an HC-05 Bluetooth module, motor drivers, and multiple high-power DC motors, all powered by a central 5V power supply. The camera sends visual data to the Raspberry Pi, which serves as the main controller, while the HC-05 Bluetooth module allows remote commands to be sent wirelessly from a smartphone or computer.

The Raspberry Pi interprets these commands and uses its GPIO pins to control various motors through the motor driver. Specifically, GPIO pins 26, 19, 13, and 6 control the robot's movement (forward, backward, left, and right), pins 2 and 3 operate the gripper for opening and closing, pins 21 and 20 control the lifting arm's up and down motions, and pins 17 and 27 handle additional actuator actions like moving or dropping items. Functions defined in the Python code activate specific GPIO combinations to execute tasks such as picking up colored boxes, transporting them, and dropping them at predefined locations, with each box type (green, red, or blue) having a distinct set of motor commands and timing sequences for coordinated movement.

Robotic arm based on object sorting with bluetooth command

The HC-05 communicates with the Raspberry Pi through its UART pins, typically GPIO14 and GPIO15, enabling command reception and initiating the appropriate motor-driven routines coded in Python.

3.2 Working & Connections

1. GPIO Pin Connections

The Raspberry Pi controls multiple motors via GPIO pins, categorized into three functional groups:

A. Driving Motors (Robot Movement)

These are connected to four GPIO pins to control the direction (FORWARD, BACKWARD, LEFT, RIGHT):

Motor Function GPIO Pin Variable

Left Motor IN1 GPIO 26 IN1

Left Motor IN2 GPIO 19 IN2

Right Motor IN3 GPIO 13 IN3

Right Motor IN4 GPIO 6 IN4

These four control two H-Bridge controlled motors, allowing for bidirectional movement.

B. Servo Motor or Lift Arm Motor

These are used for vertical movement (lifting/lowering an arm):

Function GPIO Pin Variable

Up GPIO 21 IN7

Down GPIO 20 IN8

These likely control a DC gear motor or actuator for lifting objects.

C. Gripper (Open/Close)

Used to grip or release the object:

Function GPIO Pin Variable

Open GPIO 2 IN5

Close GPIO 3 IN6

Robotic arm based on object sorting with bluetooth command

Also likely driven by a small DC motor using an H-bridge.

D. Slide or Pusher Mechanism

Used to push or drop boxes into location:

Function	GPIO Pin	Variable
Forward (Push)	GPIO 17	IN9
Backward (Retract)	GPIO 27	IN10

This seems to move a slider for final placement after detection.

2. Camera Setup (Color Detection)

Your code uses `cv2.VideoCapture(0)` to activate a USB camera for detecting colors using **OpenCV**.

- **HSV filtering** is applied to separate red, green, and blue.
- Contours are found and highlighted using `cv2.findContours()`.

Each color detection script counts occurrences, and once >50 detections are confirmed, it triggers movement functions accordingly.

3. Bluetooth Communication

You use the **serial** module to receive commands over **Bluetooth** from `/dev/ttyAMA0`.

Example Commands:

Bluetooth Data	Action	Triggered Function
rr	Red Box, Red Det.	<code>red_box()</code>
gb	Green Box, Blue Det	<code>blue_box()</code>
f	Move Forward	<code>FORWORD()</code>
s	Stop Movement	<code>STOP()</code>
u	Lift Arm Up	<code>up()</code>
d	Lower Arm	<code>down()</code>
o	Open Gripper	<code>Open()</code>
c	Close Gripper	<code>close()</code>

4. High-Level Workflow

A. Detection and Sorting:

1. **Bluetooth** receives a command (e.g., rr).
2. It launches a corresponding detection script (e.g., red_detection.py).
3. Camera detects the color using OpenCV.
4. If the object is detected:
 - The robotic arm lowers.
 - The gripper opens → closes to pick the object.
 - Arm lifts the object.
 - Robot moves to a defined position.
 - Box is dropped using drop() or move().

B. Return to Base:

After placing the box, the robot returns using:

- original() or original2() → retract slide
- BACKWORD() → return to starting point

CHAPTER 4

HARDWARE & SOFTWARE

4.1 Hardware Setup:

- Support Raspberry Pi 3/4 (preferable)
- webCam
- Bluetooth device HC-05
- Battery(12 volt)
- Motor Drivers
- DC Motors (High Power)
- ARM

4.2 Software Development:

This project is dependent on the following packages:

- Python

4.2.1 Python:

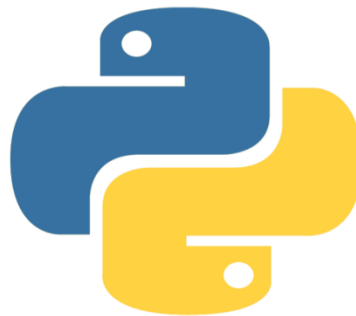


Fig 4.1 python

Robotic arm based on object sorting with bluetooth command

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Python is dynamically type-checked and garbage-collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming. It is often described as a "batteries included" language due to its comprehensive standard library. Guido van Rossum began working on Python in the late 1980s as a successor to the ABC programming language and first released it in 1991 as Python 0.9.0. Python 2.0 was released in 2000. Python 3.0, released in 2008, was a major revision not completely backward-compatible with earlier versions. Python 2.7.18, released in 2020, was the last release of Python 2. Python consistently ranks as one of the most popular programming languages, and has gained widespread use in the machine learning community.

4.3 Components

1. Mechanical Structure:

Frame and Joints: The physical framework of the robotic arm, typically constructed from materials like aluminum or high-strength plastic, provides structural integrity. The arm consists of multiple segments connected by joints, allowing movement across various axes.



Fig 4.2 Robotic Arm

2. Raspberry Pi 3

Image Processing and Object Recognition: The Raspberry Pi, equipped with a camera module, can perform complex image processing tasks to identify and classify objects based on attributes like color and size. Utilizing libraries such as OpenCV, the Raspberry Pi processes visual data to determine the object's characteristics and convey this information to the Arduino for action.

The **Raspberry Pi 3** is a small, affordable, **single-board computer** developed by the Raspberry Pi Foundation. It is designed to teach programming, electronics, and computing concepts, and it's widely used in DIY electronics, robotics, and IoT projects.

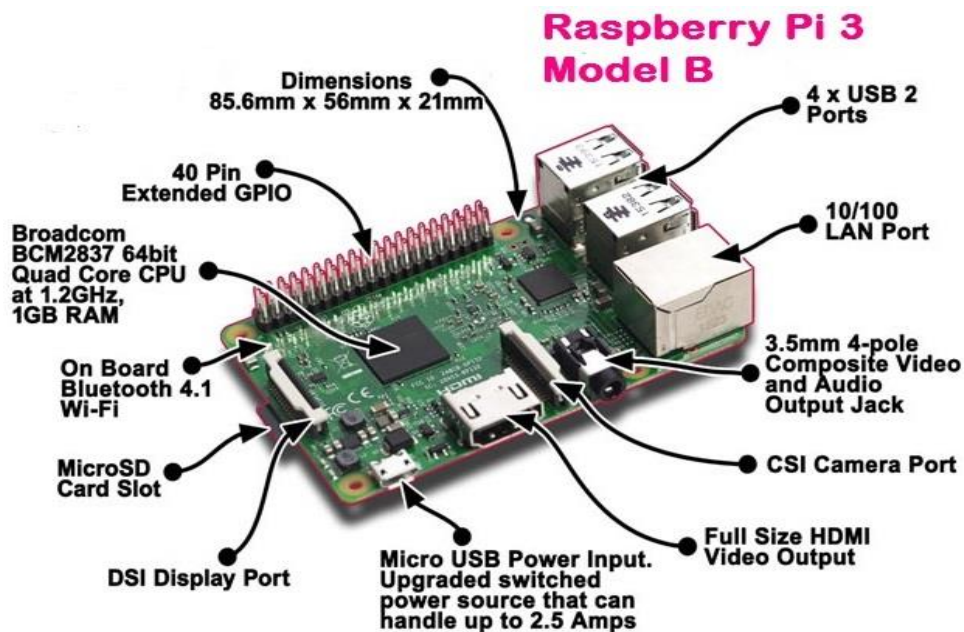


Fig 4.3 Raspberry Pi 3

Technical Specifications:

- Broadcom BCM2837 64bit Quad Core Processor powered Single Board Computer running at 1.2GHz 1GB RAM
- BCM43438 WiFi on board

Robotic arm based on object sorting with bluetooth command

- Bluetooth Low Energy (BLE) on board
- 40pin extended GPIO
- 4 x USB 2 ports
- 4 pole Stereo output and Composite video port
- Full size HDMI CSI camera port for connecting the Raspberry Pi camera
- DSI display port for connecting the Raspberry Pi touch screen display
- Micro SD port for loading your operating system and storing data
- Upgraded switched Micro USB power source (now supports up to 2.4 Amps)
- Same form factor as the Pi 2 Model B, however the LEDs have changed position

3. Jumper Wires



Fig 4.4 Jumper wires

A **jump wire** (also known as jumper, jumper wire, jumper cable, DuPont wire, or DuPont cable) is an electrical wire or group of them in a cable with a connector or pin at each end (or sometimes without them – simply "tinned"), which is normally used to interconnect the components of a breadboard or other prototype or test circuit, internally or with other equipment or components, without soldering.

Individual jump wires are fitted by inserting their "end connectors" into the slots provided in a breadboard, the header connector of a circuit board, or a piece of

4. MOTOR DRIVER CIRCUIT:

We are employing here L293D motor driver. In the design of the motor driver circuit we had considered the following points:

- The required direction of rotation.
- The current and voltage requirement.
- The interfacing with the logic circuit.

L293D is a motor driver integrated circuit (IC). Motor drivers act as current amplifiers since they take a low-current control signal and provide a higher-current signal. This higher current signal is used to drive the motors.

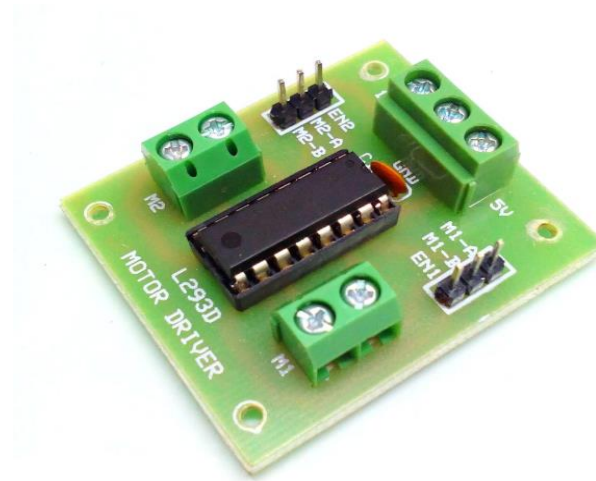


Fig 4.5 Motor Driver

In its common mode of operation, two DC motors can be driven simultaneously, both in forward and reverse direction. The motor operations of two motors can be controlled by input logic at pins 2 & 7 and 10 & 15. Input logic 00 or 11 will stop the corresponding motor. Logic 01 and 10 will rotate it in clockwise and anticlockwise directions, respectively.

5. DC Motor:

A DC motor is any of a class of rotary electrical machines that converts direct current electrical energy into mechanical energy. The most common types rely on the forces produced by magnetic fields. Nearly all types of DC motors have some internal mechanism, either electromechanical or electronic; to periodically change the direction of current flow in part of the motor. DC motor has two wires, we can say them positive terminal and negative terminal, when these wires are connected with power supply the shaft rotates. We can reverse the direction of the rotation. L293d chip is very safe to use for DC motor control. This L293D is 16bit chip. Chip is design to control four DC motor, there are two inputs and two outputs for each motor.

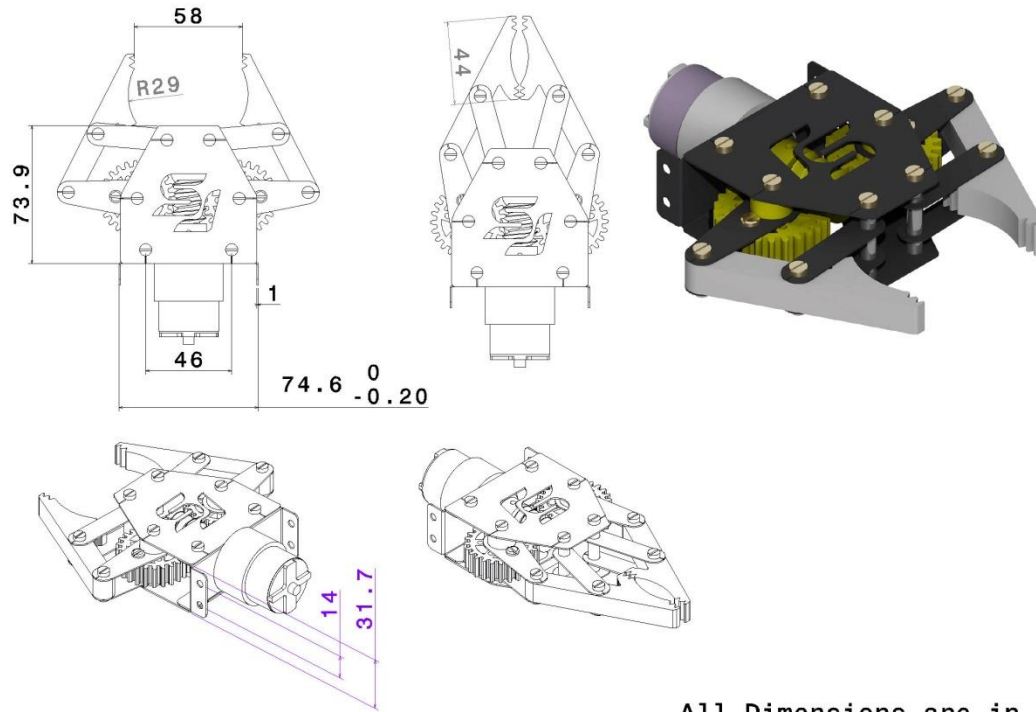


Fig 4.6 DC Motor

There are two Enable pins on L293D. Pin 1 and pin 9, for being able to drive the motor, the pin 1 and 9 need to be high. For driving the motor with left H-bridge we need to enable pin 1 to high. And for right H-Bridge we need to make the pin 9 to high. If anyone of the either pin1 or pin9 goes low then the motor in the corresponding section will suspend working. It's like a switch.

6. Pick and Place ARM:

- **GRIPPER ARM:** The gripper module is state of the art robotic arm which can be used in various 'pick and place' kind of robots. It works on DC Motor (9 to 12V DC).
- Change in rotation direction of the DC Motor, generates Jaw Open & Close Action.
- The DC motor can be easily be controlled with the help of DPDT Switch (manual mode) or with the help of any micro controller along with L293D Motor Driver module.



All Dimensions are in mm.

Fig 4.7 Gripper orthogonal view, main view

- **LIFTER ASSEMBLY:** It is made from laser cut Metal and acrylic. There is a worm gear and spur gear assembly which is attached with a DC motor (9 to 10 volt) to provide torque so that gripper can pick and lift the load.

•

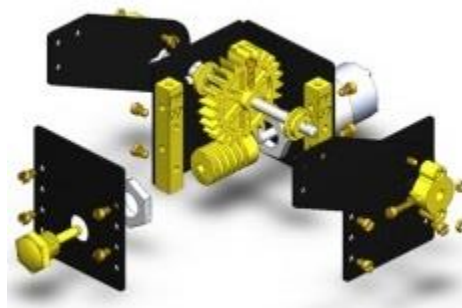


Fig 4.8 Lifter Parts

LIFTER PARTS:

- Gripper assembly Plates.

Robotic arm based on object sorting with bluetooth command

- Fiber Grippers-2nos.
- 45 RPM Motor-1nos.
- Worm Gear-1nos.
- Spur Gear-2nos.
- Different Screws and nuts.

LIFTER ASSEMBLY: It is made from laser cut Metal and acrylic. There is a worm gear and spur gear assembly which is attached with a DC motor (9 to 10 volt) to provide torque so that gripper can pick and lift the load

LIFTER PARTS:

- Gripper assembly Plates.
- Fiber Grippers-2nos.
- 45 RPM Motor-1nos.
- Worm Gear-1nos.
- Spur Gear-2nos.
- Different Screws and nuts.

7. POWER SUPPLY: To provide energy to DC motors for movement of robot A Battery of DC (6 volt to 12 V, 1.3Ah) is being used.



Fig 4.9 power supply Driver



Fig 4.10 A 12v Power supply

Motion Control: The Arduino microcontroller is responsible for controlling the robotic arm's movements. It receives commands from the Raspberry Pi and translates them into precise motions by actuating servos or motors, enabling the arm to pick and place objects accurately.

Bluetooth Communication: By integrating a Bluetooth module (such as the HC-05) with the Raspberry Pi, the system can receive wireless commands from external devices like smartphones or computers, allowing for manual control or reprogramming of tasks.

8. WebCam

Use: Captures real-time video or images for image processing (e.g., object recognition).

Pin Format:

- Connects to Raspberry Pi's **CSI camera port**
- **Powered via 5V and GND**
- Uses **MIPI-CSI interface**, not GPIO pins



Fig 4.11 web cam

Technical Specifications:

1. General Information

- **Product Name:** (e.g., Logitech C920 HD Pro Webcam)
- **Model Number:** (e.g., C920)
- **Manufacturer:** (e.g., Logitech)
- **Type:** USB external webcam / built-in laptop webcam

2. Video Performance

- **Maximum Resolution:** 1920 x 1080 (Full HD) / 4K / 720p
- **Frame Rate:** 30 FPS @ 1080p / 60 FPS @ 720p (depending on model)
- **Lens Type:** Glass lens / Plastic lens
- **Field of View (FOV):** 78° / 90° / Adjustable
- **Autofocus:** Yes / No
- **Low Light Correction:** Yes / No
- **Zoom Capability:** Digital Zoom (up to Xx)

3. Audio

- **Microphone Type:** Built-in stereo / mono
- **Number of Microphones:** 1 / 2
- **Noise Reduction:** Yes / No

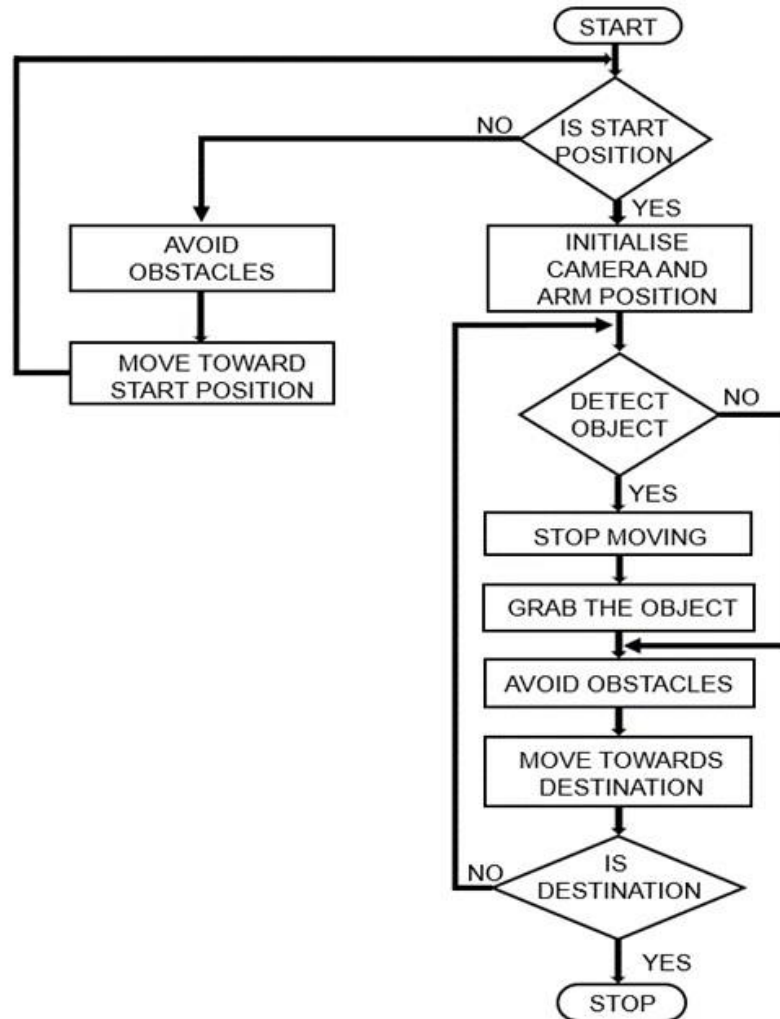
4. Connectivity

- **Interface:** USB 2.0 / USB 3.0 / USB-C
- **Cable Length:** X meters (e.g., 1.5m)

Chapter 5

SYSTEM FLOW CHART

5.1 Flow Diagram:



5.1 Flow Chart

1. Start

The robot begins its operation.

2. Is Start Position?

The system first checks whether the robot is at its predefined **starting position**.

- **If YES:** It proceeds to initialize the camera and arm.

Robotic arm based on object sorting with bluetooth command

- **If NO:** It enters a subroutine to **avoid obstacles** and move toward the start position before rechecking.

3. Initialize Camera and Arm Position

Once at the start position, the robot:

- Activates the **camera system** for object detection.
- Sets the **robotic arm** to its default starting posture, ready to grab an object.

4. Detect Object

The camera and detection algorithm (likely using OpenCV in HSV space) begin scanning the area for a **color-matched object**.

- **If NO object is detected:** The robot keeps scanning in a loop.
- **If YES:** It stops its current movement to avoid losing the target.

5. Stop Moving

Once the object is confirmed, the robot halts all locomotion to prepare for precise interaction.

6. Grab the Object

The robotic arm or gripper mechanism is activated to **pick up** the detected object.

7. Avoid Obstacles

With the object secured, the robot checks for and navigates around any **obstacles** in its path toward the sorting destination.

8. Move Toward Destination

The robot begins moving toward the predefined **drop-off location** (e.g., red bin, green bin, etc.), guided by either pre-programmed directions or real-time navigation.

9. Is Destination?

It constantly checks whether it has reached the correct sorting bin.

- **If NO:** It continues moving and avoiding obstacles.
- **If YES:** It proceeds to stop.

10. Stop

Once the object is delivered to the destination, the robot **completes the cycle** and halts.

FLOWCHART OF CAMERA MODULE:

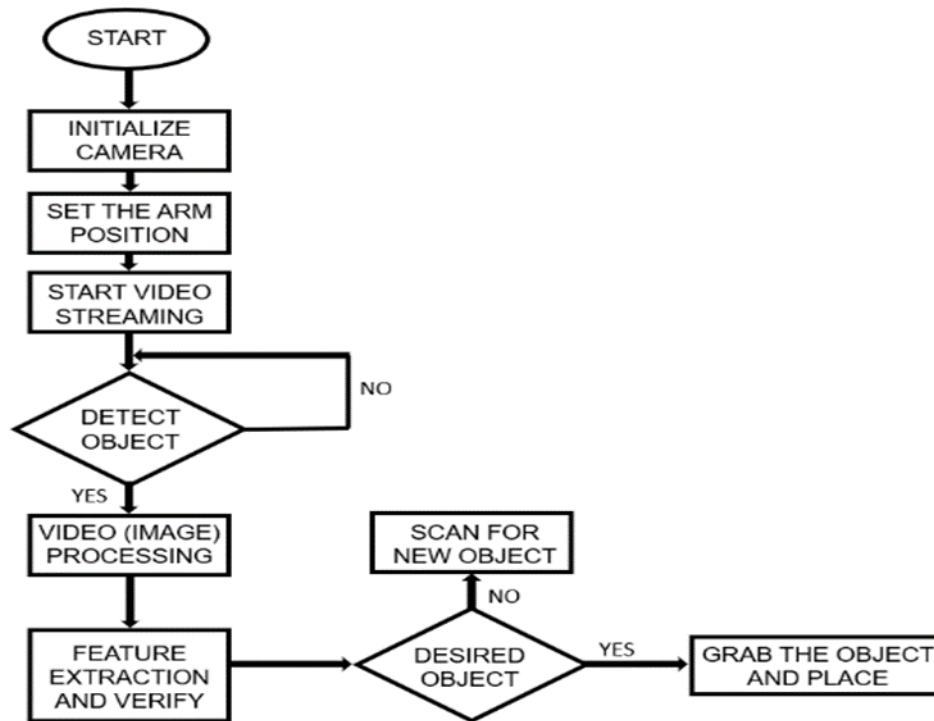


Fig 5.2 Flow chart

The working principles of proposed project is as follows:

- The source and destination co-ordinates which is been obtained manually is fed to microcontroller using Bluetooth module.
- GPS module compares the current co-ordinate of robot with target (source) co-ordinate and sends the difference of both to microcontroller that drives the DC motors till robot reaches source.
- Once robot reaches the source , the camera module which is an eye of the robot captures the picture of objects and sends it to microcontroller , if the image of object matches with the pre-defined image than microcontroller drives the servo motors to grab that particular object.
- After grabbing the object the robot is driven towards destination with the help of GPS module.
- After reaching the destination it places the desired object.
- Throughout the navigation process the robot uses ultrasonic sensor to avoid the obstacles and finds shortest distance to move further.

5.2 Algorithm & Mathematical Concepts

The robot system follows a **step-by-step algorithm** that combines color detection, motor control, and Bluetooth communication:

- **InitializationPhase:**
The Raspberry Pi sets up all GPIO pins for motor control and disables any warnings. Motors are assigned to specific pins for functions such as movement, gripper operation, and arm lifting.
- **WaitingforCommand:**
The robot listens for commands sent over Bluetooth (via serial communication). Each command corresponds to a specific color or action (e.g., 'rg' means sort a red object into the green bin).
- **ColorDetectionPhase:**
Based on the received command, the Raspberry Pi executes a corresponding Python script (`red_detection.py`, `green_detection.py`, or `blue_detection.py`) which:
 - Captures live video using OpenCV.
 - Converts the image from **BGR to HSV** color space.
 - Applies a **color mask** to isolate the desired color range.
 - Uses **contour detection** to find the object with sufficient area in the frame.
 - Confirms detection after a threshold number of frames.
- **SortingOperation:**
Once the object is detected:
 - The robot lowers the gripper (arm down), opens it, grabs the object, and lifts it (arm up).
 - It rotates or shifts position if needed (based on destination bin).
 - It moves forward for a specified duration.
 - It drops the object and returns to its original position.
- **ResetState:**
After sorting, the robot resets all motors to be ready for the next command.

5.2.1 Mathematical Concepts Used:

- **HSV Color Space Filtering:**
 - HSV stands for Hue, Saturation, and Value.
 - HSV makes it easier to isolate colors because hue represents the color type, making it more robust to lighting changes than RGB.
 - Mathematical thresholds are applied to define lower and upper bounds of a color (e.g., red = [0, 100, 0] to [10, 255, 60]).
 - `cv2.inRange()` is used to apply these thresholds, producing a binary mask.
- **Morphological Operations:**
 - `cv2.dilate()` uses a **convolution kernel** (matrix of 1s) to emphasize the masked regions.
 - This helps fill small gaps and ensure contours are detected as a continuous shape.
- **Contour Detection and Area Calculation:**

Robotic arm based on object sorting with bluetooth command

- `cv2.findContours()` returns a set of shapes found in the binary mask.
- `cv2.contourArea()` computes the area of each detected shape using pixel counts.
- If the area > 300 (a threshold), it's considered a valid object to sort.
- **Kinematic Motion Using Timed Control:**
 - Motor movements (forward, backward, arm up/down) are controlled using `time.sleep()` to approximate distance or rotation.
 - Although not mathematically precise (no feedback loop), the assumption is **distance = speed × time** where speed is constant (based on motor voltage).
 - This basic kinematic principle allows estimation of how far the robot or arm moves based on fixed timing.

5.3 Implementation

The project is a Raspberry Pi-based autonomous sorting robot designed to detect the color of an object and place it into the correct bin. The robot uses several DC motors, controlled through the Raspberry Pi's GPIO pins, to move around, pick up items using a gripper, lift and lower its arm, and rotate or shift its base to align with different bins. The robot begins by initializing all the necessary GPIO pins, setting them as outputs, and ensuring all motors are initially off.

There are different motors connected to the Raspberry Pi: two motors for driving the wheels (controlled via IN1 to IN4), one motor for opening and closing the gripper (IN5 and IN6), another motor to lift and lower the arm (IN7 and IN8), and finally, one more motor to shift or rotate the robot's base left or right (IN9 and IN10). The wheel motors allow the robot to move forward, backward, left, or right. The arm motor allows the robot to pick up and place objects vertically, while the gripper motor controls the grip on objects. The final motor is used to slightly shift the robot's position or orientation before or after placing an object into a bin.

The robot receives commands via Bluetooth using a serial interface. Based on the received command—for example, 'rg' which might mean “red item to green bin”—the Raspberry Pi triggers the appropriate color detection script, such as `red_detection.py`. This script uses OpenCV and a webcam to detect specific colors in front of the robot. It converts the video feed into HSV color space and applies masking to isolate the desired color. When the color is

Robotic arm based on object sorting with bluetooth command

detected consistently over several frames, the script exits and the robot proceeds to the sorting action.

5.4 Mathematical Concepts

The robot system follows a **step-by-step algorithm** that combines color detection, motor control, and Bluetooth communication:

- **InitializationPhase:**
The Raspberry Pi sets up all GPIO pins for motor control and disables any warnings. Motors are assigned to specific pins for functions such as movement, gripper operation, and arm lifting.
- **WaitingforCommand:**
The robot listens for commands sent over Bluetooth (via serial communication). Each command corresponds to a specific color or action (e.g., 'rg' means sort a red object into the green bin).
- **ColorDetectionPhase:**
Based on the received command, the Raspberry Pi executes a corresponding Python script (`red_detection.py`, `green_detection.py`, or `blue_detection.py`) which:
 - Captures live video using OpenCV.
 - Converts the image from **BGR to HSV** color space.
 - Applies a **color mask** to isolate the desired color range.
 - Uses **contour detection** to find the object with sufficient area in the frame.
 - Confirms detection after a threshold number of frames.
- **SortingOperation:**
Once the object is detected:
 - The robot lowers the gripper (arm down), opens it, grabs the object, and lifts it (arm up).
 - It rotates or shifts position if needed (based on destination bin).
 - It moves forward for a specified duration.
 - It drops the object and returns to its original position.
- **ResetState:**
After sorting, the robot resets all motors to be ready for the next command.

5.3.1 Mathematical Concepts Used:

- **HSV Color Space Filtering:**
 - HSV stands for Hue, Saturation, and Value.
 - HSV makes it easier to isolate colors because hue represents the color type, making it more robust to lighting changes than RGB.
 - Mathematical thresholds are applied to define lower and upper bounds of a color (e.g., `red = [0, 100, 0] to [10, 255, 60]`).
 - `cv2.inRange()` is used to apply these thresholds, producing a binary mask.
- **Morphological Operations:**

Robotic arm based on object sorting with bluetooth command

- `cv2.dilate()` uses a **convolution kernel** (matrix of 1s) to emphasize the masked regions.
- This helps fill small gaps and ensure contours are detected as a continuous shape.
- **Contour Detection and Area Calculation:**
 - `cv2.findContours()` returns a set of shapes found in the binary mask.
 - `cv2.contourArea()` computes the area of each detected shape using pixel counts.
 - If the area > 300 (a threshold), it's considered a valid object to sort.
- **Kinematic Motion Using Timed Control:**
 - Motor movements (forward, backward, arm up/down) are controlled using `time.sleep()` to approximate distance or rotation.
 - Although not mathematically precise (no feedback loop), the assumption is **distance = speed × time** where speed is constant (based on motor voltage).
 - This basic kinematic principle allows estimation of how far the robot or arm moves based on fixed timing.

CHAPTER 6

ADVANTAGES AND DISADVANTAGES

6.1 Advantages

1. **Autonomous Color Sorting**
 - Automatically detects red, green, and blue objects and performs specific actions to sort them accordingly.
2. **Modular and Reusable Code**
 - Functional approach (e.g., `FORWORD()`, `green_box()`, etc.) makes it easy to maintain, reuse, and extend.
3. **Real-time Control via Bluetooth**
 - Supports real-time command input using Bluetooth, allowing remote control.
4. **Integration with OpenCV**
 - Utilizes OpenCV for effective color-based object detection.
5. **Hardware Control via GPIO**
 - Interfacing with motors and actuators using GPIO enables physical actions like lifting, dropping, opening, and closing.
6. **Multi-color Detection Logic**
 - HSV-based color segmentation is robust for varying lighting conditions compared to RGB.
7. **Basic Error Avoidance**
 - Threshold checks (e.g., if $b > 50$) reduce false positives in detection.

6.2 Disadvantages

1. **No Error Handling or Fault Tolerance**
 - No try/except blocks or hardware status checks. If a motor or sensor fails, the whole system may hang or behave unexpectedly.
2. **Hardcoded Timings**
 - Time-based control (e.g., `time.sleep(5)`) is not dynamic; changes in real-world conditions could affect accuracy.
3. **Lack of Scalability**
 - Limited to three colors and fixed logic; adding more colors or objects requires manual coding.
4. **Blocking Operations**
 - Commands like `time.sleep()` block the program, making it less responsive during those intervals.
5. **No Feedback Loop**
 - No sensor or vision feedback after moving the object — it assumes operations complete correctly.
6. **Code Repetition**
 - Some detection code is duplicated across color scripts instead of being modularized.
7. **Dependence on Lighting Conditions**

- Despite using HSV, the detection system may still misbehave under very poor lighting or reflections.

6.3 Limitations

1. **Limited Color Range Detection**
 - Only red, green, and blue are handled. Any non-primary colors (e.g., yellow or purple) would be ignored or misclassified.
2. **Single Camera, Fixed Position**
 - A fixed webcam angle may limit detection accuracy and object visibility.
3. **Single-Threaded Execution**
 - The program runs in a blocking manner. While executing one command, it can't process new inputs or perform other tasks.
4. **No Object Differentiation by Shape or Size**
 - All objects of the same color are treated equally, regardless of size, shape, or position.
5. **Requires Manual Restart or External Supervision**
 - If the robot fails or hangs, manual intervention is likely needed due to no watchdog or self-recovery logic.
6. **Basic Bluetooth Security**
 - No authentication or encryption on Bluetooth commands — it could be vulnerable to spoofing.

CHAPTER 7

RESULT & FUTURE SCOPE

7.1 RESULT

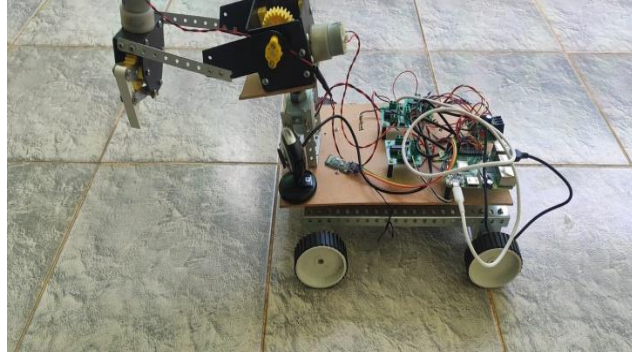


Fig7.1 Result A initial position

The implemented color-sorting robot functions effectively by detecting red, green, and blue colored objects using real-time camera input and sorting them into respective bins through coordinated motor movements. The system successfully interprets Bluetooth commands, processes live video through OpenCV for color recognition, and controls various DC motors via GPIO pins on the Raspberry Pi to carry out gripping, lifting, movement, and dropping operations. Testing has shown that the robot can consistently recognize colors, pick up objects, and deposit them accurately into predefined locations, validating the system's design and functionality.

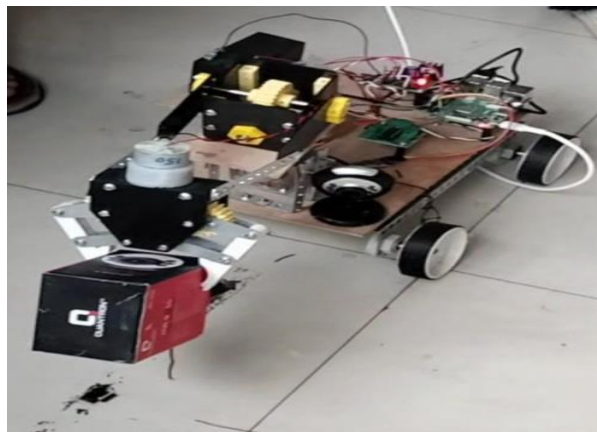


Fig 7.2 Result B object picking



Fig7.3 Result C Drop to Destination

The implemented color-sorting robot functions effectively by detecting red, green, and blue colored objects using real-time camera input and sorting them into respective bins through coordinated motor movements. The system successfully interprets Bluetooth commands, processes live video through OpenCV for color recognition, and controls various DC motors via GPIO pins on the Raspberry Pi to carry out gripping, lifting, movement, and dropping operations. Testing has shown that the robot can consistently recognize colors, pick up objects, and deposit them accurately into predefined locations, validating the system's design and functionality.

7.2 FUTURE SCOPE

This project has significant potential for expansion and improvement. In the future, the system can be enhanced with ultrasonic or infrared sensors to allow for obstacle detection and autonomous path planning, making the robot more adaptable in dynamic environments. Machine learning algorithms could be integrated to improve color recognition accuracy under varying lighting conditions or to recognize more complex object features like shape or texture. The gripper mechanism could also be upgraded with servos for more precise control, and the entire robot can be placed on a mobile chassis to perform sorting in larger or distributed workspaces. Furthermore, IoT integration could enable remote monitoring and control of the sorting process, making it suitable for industrial or smart warehouse applications.

CHAPTER 8

Conclusions

In conclusion, this project successfully demonstrates an intelligent robotic system capable of detecting object colors and sorting them into designated bins using a Raspberry Pi. By integrating computer vision through OpenCV, precise motor control using GPIO outputs, and wireless communication via Bluetooth, the robot can autonomously identify red, green, or blue objects and perform the required physical movements to sort them correctly. The system showcases how embedded hardware and software can work together to solve real-world automation tasks, laying a strong foundation for further development in robotic sorting, smart automation, and AI-assisted control systems.